

JSQKV: Joint Sparsification and Quantization for KV-Cache Compression and Decode Acceleration

Hao Zhang¹[0009-0001-1044-0312], Xiaoli Gong¹[0009-0001-1044-0312]*, Haoran Li¹[0009-0001-1044-0312], Huayou Su²[0009-0001-1044-0312], Qingxia Chen³[0009-0001-1044-0312], and Jin Zhang¹[0009-0001-1044-0312]

¹ College of Computer Science, Nankai University, Tianjin, China
{2120230710, gongxiaoli, wanch_lhr, nkzhangjin}@nankai.edu.cn

² National University of Defense Technology, Changsha, China
shyou@nudt.edu.cn

³ Qian Xuesen Laboratory of Space Technology, Beijing, China
chenqingxia_0882@163.com

Abstract. As long-context large language models become increasingly widespread, autoregressive decoding is increasingly bottlenecked by KV-cache footprint and memory bandwidth. Existing KV-cache compression methods typically rely on sparsification or low-bit quantization, but simply combining the two often fails to simultaneously preserve model quality and deliver real end-to-end speedup, since sparsification changes how compressed KV-cache states are represented and executed during decoding. To address this issue, we present JSQKV a joint sparse-quant framework for KV-cache compression and decode acceleration that co-designs compression policy, compressed format, and execution path. Specifically, JSQKV combines attention-score-based differential sparsification with a dual-window online mechanism, Per-Token-Tile quantization with Hadamard rotation, and a bitmap-based sparse-quant format with a matching decode kernel. Through this joint design, compressed KV-cache data remains compact in memory and is more effectively translated into reduced memory traffic and end-to-end speedup. Experimental results show that, under matched compression budgets, JSQKV generally outperforms the sequential MUSTAFAR+KIVI baseline on representative LongBench tasks, with larger gains under more aggressive sparse-quant settings. On Llama-3-8B-Instruct, JSQKV improves the average score on six representative LongBench tasks from 39.83 to 43.12, while also improving end-to-end throughput by 44.2% over dense decoding and increasing the KV-cache compression ratio to $3.48\times$.

Keywords: KV-cache compression · Sparsity · Quantization

1 Introduction

As context windows continue to grow in large language models, autoregressive decoding is increasingly bottlenecked by KV-cache footprint and memory bandwidth [7]. Since the KV cache grows linearly with context length and is re-

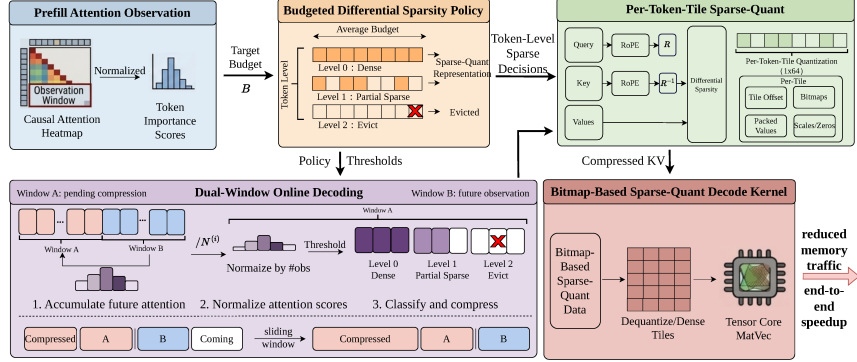


Fig. 1. Overview of JSQKV. The framework combines differential sparsity, dual-window online execution, Per-Token-Tile quantization with Hadamard rotation, and a bitmap-based sparse-quant decode path.

peatedly accessed during decoding, this stage is more memory-bound than prefill and increasingly limits throughput, batch size, and deployment efficiency in long-context settings [14, 1].

Prior work on KV-cache compression mainly follows two directions: sparsification and low-bit quantization. Token eviction and sparsification reduce retained history or prune redundant KV states [23, 27, 15, 4, 24, 17, 13], while quantization lowers the numerical precision of retained cache elements [16, 10, 2, 5]. Although both are effective, existing studies still largely treat them as separate components.

In practice, joint sparse-quant KV-cache compression is not a simple sparse-then-quantize pipeline. Sparsification changes not only how much KV data is kept, but also the retained token and feature layout, which in turn affects quantization granularity, compressed data layout, and decode-time execution. As a result, algorithm-level compression gains do not automatically translate into lower memory traffic or real end-to-end speedup during decoding [1, 28, 13]. Moreover, token-differential compression is easy to estimate in prefill, where future attention is observable, but difficult to apply online during autoregressive decoding, where a token must be cached before its future importance can be known. Thus, the key challenge is twofold: preserving model quality under aggressive compression and keeping the compressed representation directly executable for practical decode-time acceleration.

To address these challenges, we propose JSQKV, a joint sparse-quant framework for KV-cache compression and decode acceleration. Rather than treating sparsification and quantization as separate post-processing steps, JSQKV co-designs compression policy, compressed representation, and runtime execution in a unified decode-stage pipeline, as illustrated in Fig. 1. Section 3.1 introduces an attention-score-based differential sparsification policy that allocates different compression budgets to historical tokens according to estimated future impor-

tance. Since such token-differential decisions are easy to estimate in prefill but not directly available when tokens are generated online, Section 3.2 presents a dual-window mechanism that accumulates future observations before compression and makes the differential policy applicable during autoregressive decoding. After sparse decisions are made, the remaining KV states still require a low-bit representation that is both robust and compatible with token-level sparse execution. To this end, Section 3.3 introduces Hadamard-based Per-Token-Tile quantization to mitigate key outliers and align quantization granularity with downstream computation. Finally, Section 3.4 presents a bitmap-based sparse-quant format and a matching decode kernel that keep the KV cache compact in memory and convert compression savings into reduced memory traffic and practical end-to-end decoding speedup. Together, these components form a unified sparse-quant decode pipeline for accurate and efficient KV-cache compression, and our evaluations show that JSQKV compares favorably with sequential sparse-quant baselines while also delivering practical end-to-end decoding speedup.

The main contributions of this paper are as follows:

- We propose an attention-score-based differential sparsification policy that assigns different sparsity levels to historical tokens under a fixed average compression budget, instead of applying a uniform compression ratio to all tokens.
- We design a dual-window online execution mechanism that makes this differential policy usable in autoregressive decoding, where future attention evidence is unavailable when a token is first generated.
- We propose Hadamard-based Per-Token-Tile quantization for the retained KV states to stabilize low-bit representation and align quantization granularity with token-level compression decisions.
- We design a bitmap-based sparse-quant format and matching decode kernel so that compressed KV-cache representations can be translated into reduced memory traffic and practical decode-time speedup.

2 Background and Related Work

KV Sparsification KV-cache sparsification reduces memory cost by removing less useful history or pruning retained states during decoding. Token-level retention and eviction methods, such as StreamingLLM [23], H₂O [27], SnapKV [15], and PyramidKV [4], exploit the unequal contribution of historical tokens to future decoding. Finer-grained sparsification further explores redundancy within retained KV states. ThinK [24] prunes key channels using query-dependent importance, KVPruner [17] studies structured pruning of KV blocks, and MUSTA-FAR [13] shows that unstructured per-token sparsity can be practical when paired with a bitmap-based sparse format and a matching attention kernel.

KV-Cache Quantization KVQuant [10] studies ultra-low-bit KV-cache quantization for long-context inference, KIVI [16] uses asymmetric quantization with

different granularities for Keys and Values, and QuaRot [2] and RotateKV [5] improve low-bit robustness by reshaping activation distributions before quantization. These methods differ in quantization granularity, metadata cost, and execution compatibility.

Systems and Kernels for Attention PagedAttention [14] improves KV-cache management through block-based memory organization, while FlashDecoding [9] and FlashInfer [26] optimize decode-stage attention in memory-bound settings. Loki [19] leverages low-rank keys to reduce data movement and computation, Flash-LLM [22] supports unstructured sparsity on Tensor Cores through a load-as-sparse and compute-as-dense framework, and Coruscant [12] further co-designs GPU kernels and sparse tensor cores for efficient unstructured sparse LLM inference.

3 Method

3.1 Budgeted Differential Sparsity Policy

A fixed sparsity ratio treats all historical tokens equally, wasting budget on unimportant tokens and over-compressing important ones. To address this issue, JSQKV estimates token importance from prefill attention and instantiates a three-level sparsity policy under an explicit average-budget constraint.

We use the attention distribution in prefill to estimate the relative importance of historical tokens. Let $\mathbf{A} \in \mathbb{R}^{T_q \times T_k}$ denote the causal attention matrix of the prompt. Following the observation-window idea in SnapKV [15], we use the last L_{obs} prompt tokens as an observation window. For a prefix token $j \leq T_k - L_{\text{obs}}$, its importance score is defined as

$$s_j = \frac{T_k}{L_{\text{obs}}} \sum_{i=T_k-L_{\text{obs}}+1}^{T_k} A_{i,j}. \quad (1)$$

This score measures how much token j is attended by queries in the observation window; tokens inside the observation window are preserved by construction, and the factor T_k/L_{obs} makes scores more comparable across prompts of different lengths.

Given the importance scores $\{s_j\}$, JSQKV assigns each token to one of three compression levels. Let $[p_0, p_1, p_2]$ denote the target proportions of Level 0, Level 1, and Level 2 tokens, where

$$p_0 + p_1 + p_2 = 1. \quad (2)$$

We compute two percentile thresholds,

$$\tau_{\text{high}} = \text{Percentile}(s, 1 - p_0), \quad (3)$$

$$\tau_{\text{low}} = \text{Percentile}(s, p_2), \quad (4)$$

and assign

$$\text{level}(j) = \begin{cases} 0, & s_j \geq \tau_{\text{high}}, \\ 1, & \tau_{\text{low}} \leq s_j < \tau_{\text{high}}, \\ 2, & s_j < \tau_{\text{low}}. \end{cases} \quad (5)$$

The three levels correspond to different compression strengths. Level 0 tokens are kept dense. Level 2 tokens are fully removed from the cache. Level 1 tokens use feature-level sparsity and keep only the largest-magnitude features in their key and value vectors. For a key vector $\mathbf{k}_j \in \mathbb{R}^d$, we apply

$$\tilde{k}_{j,i} = \begin{cases} k_{j,i}, & i \in \text{TopK}(|\mathbf{k}_j|, \lfloor (1 - \rho_1)d \rfloor), \\ 0, & \text{otherwise,} \end{cases} \quad (6)$$

where ρ_1 is the feature-level sparsity ratio for Level 1 tokens. The same rule is applied to $\mathbf{v}_j$. In this way, important tokens retain more information, while less important tokens absorb more of the compression budget.

A fixed average sparsity budget does not uniquely determine the three-level policy: different dense/partial/evict allocations can share the same average compression ratio but lead to different accuracy outcomes. We therefore instantiate the policy under an explicit budget constraint. With $\rho_0 = 0$ for Level 0 and $\rho_2 = 1$ for Level 2, the target average sparsity budget B satisfies

$$B = p_0\rho_0 + p_1\rho_1 + p_2\rho_2 = p_1\rho_1 + p_2. \quad (7)$$

For a given target budget B , we search over a small feasible set of (p_0, ρ_1) pairs, recover the remaining proportions analytically, and select the final configuration on a small calibration split. This keeps the overall compression target fixed while allowing more careful allocation across dense, partially sparse, and evicted tokens.

3.2 Dual-Window Online Execution Mechanism

The policy in Section 3.1 relies on future attention, which is available in prefill but unavailable when a token is first generated during decoding. To make the same policy work online, JSQKV organizes the decode-stage KV cache into two consecutive windows of size W : Window A stores tokens waiting to be compressed, while Window B stores newly generated tokens whose queries provide additional attention observations for Window A, as illustrated in Fig. 2.

When a new query $\mathbf{q}_t$ is produced at decoding step t , we compute its attention to the tokens in Window A:

$$\alpha_t = \text{softmax}\left(\frac{\mathbf{q}_t \mathbf{K}_A^\top}{\sqrt{d}}\right), \quad (8)$$

where $\mathbf{K}_A$ denotes the key states of Window A. These scores are accumulated across subsequent decoding steps in an attention buffer $\mathbf{Acc}$, whose i -th entry

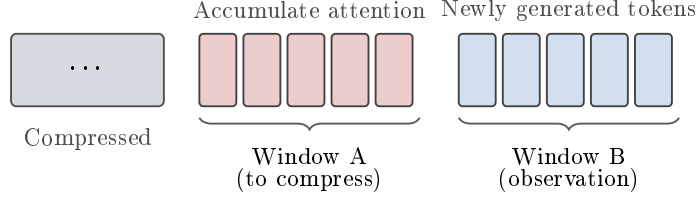


Fig. 2. Dual-window online execution during decoding. Window A accumulates attention statistics before compression, while Window B collects newly generated tokens and provides future observations.

stores the total future attention received by the i -th token in Window A while the current Window B is being filled.

Because earlier tokens in Window A are visible to more later queries, accumulated attention must be normalized before thresholding. For the token at position $i \in \{0, \dots, W-1\}$ in Window A, the number of observations is

$$N^{(i)} = 2W - 1 - i, \quad (9)$$

which gives the normalized score

$$\bar{\alpha}^{(i)} = \frac{\mathbf{Acc}[:, :, i]}{N^{(i)}}. \quad (10)$$

We then compare the normalized scores against the thresholds $(\tau_{\text{high}}, \tau_{\text{low}})$ from Section 3.1 and assign each token in Window A to Level 0, Level 1, or Level 2. The empirical stability of these prefill-estimated thresholds is examined in Appendix 1.

After classification, Level 0 tokens remain dense, Level 1 tokens are partially pruned, and Level 2 tokens are removed. The compressed Window A is merged into the historical compressed region, Window B becomes the new Window A, the accumulator is reset, and decoding continues with a new empty Window B. In this way, each token is observed before compression while the number of uncompressed tokens remains bounded by $O(W)$.

3.3 Per-Token-Tile Quantization with Hadamard Rotation

The remaining nonzero KV states still require a low-bit representation. We therefore adopt *Per-Token-Tile* quantization, which aligns the quantization unit with token-level compression decisions and the tile-based decode path while providing a fine-grained low-bit representation.

For a token vector $\mathbf{x}_t \in \mathbb{R}^d$ (either key or value), the feature dimension is partitioned into $M = d/g$ contiguous tiles of size g :

$$\mathbf{x}_t = [\mathbf{x}_{t,1}, \mathbf{x}_{t,2}, \dots, \mathbf{x}_{t,M}], \quad \mathbf{x}_{t,m} \in \mathbb{R}^g. \quad (11)$$

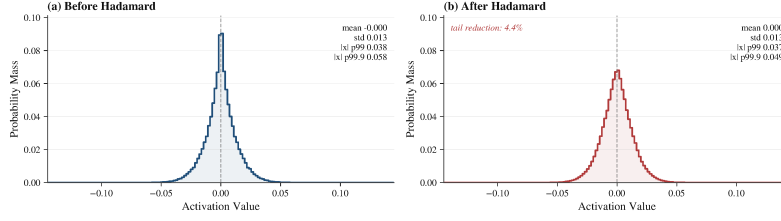


Fig. 3. Key-state activation distribution before and after Hadamard rotation on a Llama-2-7B sample. The rotation suppresses heavy tails while preserving enough variation for feature-level sparsification.

Table 1. PPL of RTN and Hadamard rotation (*Rotate*) on WikiText-2 under quantization-only and sparse-quant settings. Hadamard rotation improves low-bit quantization stability, while remaining compatible with subsequent KV sparsification.

Setting	4-bit		3-bit		2-bit	
	RTN	Rotate	RTN	Rotate	RTN	Rotate
Quant only	5.22	5.14	6.57	5.28	115.55	5.35
+50% KV sparsity	5.27	5.69	6.57	6.35	115.55	6.65

Each tile is quantized independently with its own scale $s_{t,m}$ and zero point $z_{t,m}$:

$$\mathbf{q}_{t,m} = \text{clip}\left(\text{round}\left(\frac{\mathbf{x}_{t,m} - z_{t,m}}{s_{t,m}}\right), 0, 2^b - 1\right), \quad (12)$$

where b is the quantization bitwidth.

Since key states may contain outliers that destabilize aggressive low-bit quantization, JSQKV applies Hadamard rotation to queries and keys before quantization. Let $\mathbf{H} \in \mathbb{R}^{d \times d}$ denote the normalized Hadamard matrix. We rotate

$$\tilde{\mathbf{q}}_i = \frac{1}{\sqrt{d}} \mathbf{H} \mathbf{q}_i, \quad \tilde{\mathbf{k}}_j = \frac{1}{\sqrt{d}} \mathbf{H} \mathbf{k}_j. \quad (13)$$

Because the Hadamard matrix is orthogonal, the attention score is preserved:

$$\tilde{\mathbf{q}}_i^\top \tilde{\mathbf{k}}_j = \mathbf{q}_i^\top \mathbf{k}_j. \quad (14)$$

Figure 3 and Table 1 show that Hadamard rotation suppresses heavy tails and improves low-bit robustness, especially at lower bitwidths.

3.4 Sparse-Quant Format and Decode Kernel

We co-design a sparse-quant format and a matching decode kernel so that compressed KV-cache representations reduce not only storage cost but also decode-time memory traffic.

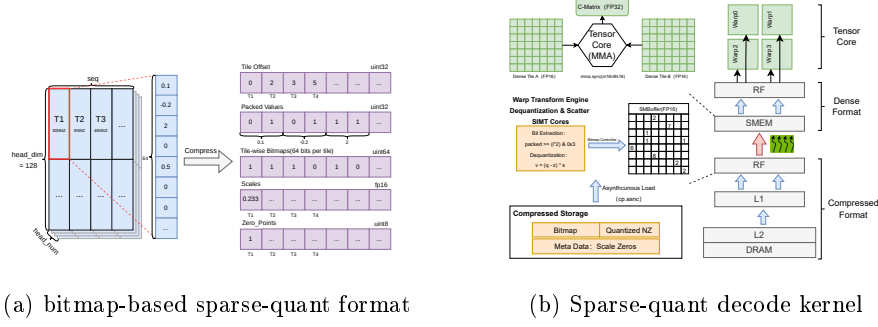


Fig. 4. Sparse-quant format and decode kernel. The format stores bitmap metadata and packed low-bit values, while the kernel follows a load-as-compressed, compute-as-dense execution path.

Figure 4 summarizes this design. Each 1×64 tile stores a bitmap for nonzero positions, packed low-bit values, a per-tile offset, and quantization metadata such as scales and zero points. During decode, the kernel follows a *load-as-compressed, compute-as-dense* strategy: it loads compressed tiles from memory, reconstructs dense shared-memory tiles through unpacking and dequantization, and then invokes Tensor Core matrix-vector computation. In this way, sparsity is handled along the loading path while the compute stage remains dense and regular.

This design is particularly useful for decode-stage attention, which is strongly memory-bound: keeping the KV cache compressed in memory reduces memory traffic, while reconstructing dense tiles only on the compute path avoids irregular accesses and control-flow divergence.

4 Experiments

4.1 Experimental Setup

Our implementation uses Python 3.10, PyTorch 2.6.0, CUDA 12.4, and custom CUDA/Triton kernels. All experiments are conducted on a single NVIDIA A100 80GB PCIe GPU with an Intel Xeon CPU and 252GB host memory.

For accuracy, we use the official LongBench [3] pipeline on six representative tasks: NarrativeQA, Qasper, MultiFieldQA-EN, HotpotQA, TREC, and LCC. We use Llama-3-8B-Instruct [8] as the main model and additionally test Llama-2-7B [20], Mistral-7B-v0.1 [11], and Qwen2.5-7B-Instruct [25]. Inputs are truncated to 8192 tokens via head-tail truncation when needed. We compare against dense decoding, a matched-budget uniform sparsity baseline based on MUSTAFAR [13], and a sequential MUSTAFAR+KIVI baseline [16] (denoted as **M+K**). Sparse-only settings use 50% and 70% average sparsity, while joint sparse-quant settings use 50/50 and 70/70 K/V sparsity with 4-bit and 2-bit

Table 2. Sparse-only results on six representative LongBench tasks for Llama-3-8B-Instruct.

Method	NarrativeQA	MultiFieldQA-			TREC	LCC	Avg.
		Qasper	EN	HotpotQA			
Dense	23.45	42.18	43.14	46.06	74.00	57.12	47.66
Uniform-50	23.44	43.73	42.93	45.94	73.50	56.03	47.59
Differential-50	23.44	43.73	42.93	45.94	74.00	57.08	47.85
Uniform-70	23.94	40.89	40.91	44.93	70.00	54.12	45.80
Differential-70	23.94	41.03	41.61	45.12	72.50	55.45	46.61

Table 3. Joint sparse-quant results on six representative LongBench tasks for Llama-3-8B-Instruct.

Setting	NarrativeQA	Qasper	MultiFieldQA-				
			EN	HotpotQA	TREC	LCC	Avg.
M+K, 50/50 + 4-bit	21.61	40.70	47.62	41.13	70.50	53.83	45.90
JSQKV, 50/50 + 4-bit	21.92	39.38	46.77	41.38	70.50	55.64	45.93
M+K, 50/50 + 2-bit	21.97	38.96	46.11	40.39	68.00	44.86	43.38
JSQKV, 50/50 + 2-bit	21.16	39.54	44.49	41.19	71.50	46.26	44.02
M+K, 70/70 + 4-bit	20.94	37.32	44.90	41.22	68.00	52.71	44.18
JSQKV, 70/70 + 4-bit	21.86	38.81	46.52	40.64	70.00	54.23	45.34
M+K, 70/70 + 2-bit	21.34	35.43	43.43	39.21	63.00	36.59	39.83
JSQKV, 70/70 + 2-bit	19.70	39.49	44.31	40.15	70.00	45.04	43.12

quantization. Differential policies are selected on a small calibration split and re-checked on a disjoint validation split before final evaluation on these tasks.

For efficiency, we use Llama-3-8B-Instruct with 4096 input tokens, 256 generated tokens, and batch sizes from 1 to 8, and report end-to-end throughput, compression overhead, compressed KV-cache size, and compression ratio. We compare dense decoding, the MUSTAFAR [13] sparse kernel, and the proposed sparse-quant runtime path.

4.2 Accuracy Results

Differential Sparsity Alone We first study differential sparsity without quantization or Hadamard rotation to isolate the effect of budget allocation, comparing JSQKV against dense decoding and a matched-budget uniform sparsity baseline at 50% and 70% average sparsity. Table 2 shows that differential sparsity remains non-negative and yields larger gains at 70% sparsity than at 50%, suggesting that token-level budget allocation becomes more valuable under tighter sparsity budgets.

Joint Sparsity-Quantization Results Table 3 reports joint sparse-quant results on six representative LongBench tasks for Llama-3-8B-Instruct [8]. Across all four settings, JSQKV achieves higher average scores than the sequential M+K

Table 4. Cross-model joint sparse-quant averages on six representative LongBench tasks.

Model	50/50 + 4-bit		50/50 + 2-bit		70/70 + 4-bit		70/70 + 2-bit	
	M+K	JSQKV	M+K	JSQKV	M+K	JSQKV	M+K	JSQKV
Llama-2-7B	31.34	31.37	31.51	30.23	30.57	30.97	29.92	30.09
Mistral-7B-v0.1	32.84	32.90	31.70	32.37	32.43	32.71	30.14	32.35
Qwen2.5-7B-Instruct	31.17	37.08	26.10	23.48	33.00	39.67	25.51	35.20

baseline, with the largest gain at 70/70 + 2-bit, where the average score improves from 39.83 to 43.12. This supports jointly aligning sparsity, quantization, and runtime execution under tighter compression budgets. A quantization-only comparison isolating the contribution of the proposed low-bit representation is reported in Appendix 2.

Table 4 shows a generally favorable but non-uniform cross-model trend over M+K: gains are more consistent under tighter compression budgets, while a few milder settings remain neutral or slightly negative. This suggests that the benefit of JSQKV mainly comes from the joint design itself, although its magnitude still depends on model family and compression setting.

4.3 Efficiency Results

In efficiency experiments, we compare JSQKV against dense decoding and the executable MUSTAFAR sparse kernel [13], since the sequential M+K baseline does not provide a unified end-to-end runtime path.

End-to-End Throughput Figure 5 reports end-to-end decoding throughput on Llama-3-8B-Instruct [8]. Dense decoding is strongest at batch size 1, where compression overhead is not yet amortized. As batch size increases, the compressed paths become more favorable, with the clearest sparse-quant advantage in the small-to-medium batch regime. Under 70/70 + 2-bit, JSQKV reaches 80.38 tokens/s at batch size 6, improving over dense decoding by 41.5% and over MUSTAFAR-70 by 3.0%. At larger batch sizes, the gap narrows, suggesting that unpacking and dequantization overheads are less effectively amortized once the workload becomes more compute-heavy.

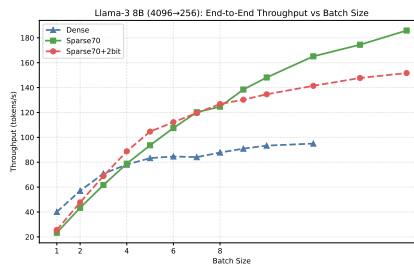


Fig. 5. End-to-end decoding throughput on Llama-3-8B. The clearest gain appears in the small-to-medium batch regime.

Table 5. Compression statistics at input length 4096.

Method	BS = 1			BS = 8		
	KV (MB)	Ratio	Time (ms)	KV (MB)	Ratio	Time (ms)
Sparse-50	2720.2	$1.51\times$	740.8	2720.2	$1.51\times$	5688.2
Sparse-70	1964.3	$2.09\times$	716.3	1964.3	$2.09\times$	5411.3
Sparse-50 + 2b	1261.2	$3.25\times$	545.2	1261.2	$3.25\times$	4317.9
Sparse-70 + 2b	1176.3	$3.48\times$	552.9	1176.3	$3.48\times$	4293.2

Compression Overhead Table 5 reports representative compression statistics on Llama-3-8B-Instruct. Compared with sparsity-only baselines, the sparse-quant path achieves higher compression ratios while keeping compression time competitive. At batch size 8, JSQKV under 70/70 + 2-bit reduces the KV cache from 1964.3MB to 1176.3MB relative to MUSTAFAR-70, improving the compression ratio from $2.09\times$ to $3.48\times$ and reducing compression time from 5411.3ms to 4293.2ms. This suggests that the more compact sparse-quant representation also lowers compression-side write-back cost.

5 Conclusion

JSQKV reduces KV-cache cost through differential sparsification, stabilized low-bit quantization, and a bitmap-based sparse-quant decode kernel. Evaluation results show that, under matched compression budgets, JSQKV generally compares favorably with a sequential sparsity-plus-quantization baseline, with larger gains under more aggressive sparse-quant settings. These results further show that compression savings can translate into practical throughput gains during decoding. Overall, these findings support treating KV-cache compression as a systems-and-algorithms co-design problem rather than as a simple composition of independent post-processing steps.

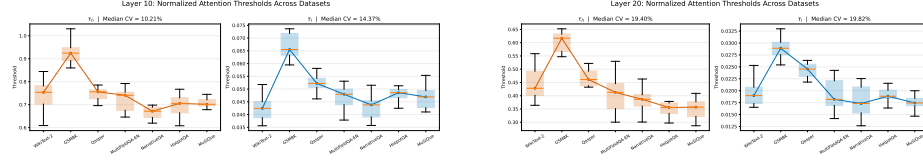
Appendix 1 Threshold Stability for Prefill-to-Decode Reuse

Section 3.1 estimates the token-importance thresholds ($\tau_{\text{high}}, \tau_{\text{low}}$) from normalized prefill attention, and Section 3.2 reuses the same thresholds during decoding after normalizing the accumulated attention by the number of available future observations. This reuse strategy relies on a simple layer-wise stability assumption: under the same normalization rule, the importance-score scale of a fixed layer should remain sufficiently consistent for prefill-estimated thresholds to remain meaningful during online decoding.

We test this assumption using the same threshold definition as JSQKV. We compute normalized attention-derived importance scores from the observed attention statistics and derive the corresponding thresholds ($\tau_{\text{high}}, \tau_{\text{low}}$). The evaluation is conducted on Llama-3-8B-Instruct [8] using seven datasets: WikiText-2 [18], GSM8K [6], MuSiQue [21], and four LongBench tasks: Qasper, Multi-

Table 6. Cross-dataset stability of the normalized attention thresholds used by the prefill-to-decode reuse path. CV is computed from the per-dataset median thresholds.

Layer	Threshold	Median range	Median CV (%)
10	τ_{high}	0.67120–0.92427	10.21
10	τ_{low}	0.04243–0.06552	14.37
20	τ_{high}	0.35568–0.61690	19.40
20	τ_{low}	0.01732–0.02890	19.82

**Fig. 6.** Cross-dataset distributions of the normalized attention thresholds used by the prefill-to-decode reuse path. Left: Layer 10. Right: Layer 20. The thresholds remain stable at a fixed layer, while their absolute scale is layer dependent.

FieldQA-EN, NarrativeQA, and HotpotQA [3]. Each dataset contributes 10 samples, and we evaluate layers 10 and 20 as representative layers.

Table 6 reports the resulting cross-dataset statistics. For Layer 10, the median CV is 10.21% for τ_{high} and 14.37% for τ_{low} ; for Layer 20, the corresponding values are 19.40% and 19.82%. Figure 6 shows the full distributions. Two observations matter most. First, for a fixed layer, both thresholds remain concentrated within a bounded range across datasets. Second, the threshold scale is clearly layer dependent, so the supported claim is reuse of *fixed per-layer thresholds*, not a single global threshold shared by all layers.

These results are sufficient for the reuse mechanism in Section 3.2. The decoder does not require decode-time scores to match prefill thresholds exactly for every sample; it only requires the normalized scores to stay on a comparable layer-wise scale. Taken together, Table 6 and Figure 6 support estimating $(\tau_{\text{high}}, \tau_{\text{low}})$ once in prefill and reusing them during decoding without step-wise re-calibration.

Appendix 2 Additional Quantization-Only Comparison

To isolate the contribution of the numerical representation in JSQKV, we compare a quantization-only instantiation of the JSQKV quantizer against a matched KIVI baseline [16]. This appendix keeps the same per-token-tile low-bit representation used by JSQKV but removes differential sparsity and the bitmap-based sparse runtime path.

Unless otherwise noted, both methods use a residual window of 128 tokens. The KIVI baseline uses channel-wise key quantization, per-token-head value

Table 7. Quantization-only comparison on representative LongBench tasks for Llama-3-8B-Instruct. Avg. is computed over the reported representative tasks.

Setting	Bit	NarrativeQA	HotpotQA	GovReport	TREC	PCount	LCC	Avg.
KIVI	4-bit	24.26	46.06	29.53	74.00	6.50	54.07	39.07
JSQKV quantizer	4-bit	22.47	46.07	30.03	73.50	5.50	57.20	39.13
KIVI	3-bit	20.81	45.64	29.35	74.00	6.75	60.94	39.58
JSQKV quantizer	3-bit	21.64	46.62	29.16	73.50	7.00	53.66	38.60
KIVI	2-bit	16.55	27.50	23.91	58.00	2.00	27.03	25.83
JSQKV quantizer	2-bit	17.90	37.93	25.63	55.50	3.00	30.37	28.39

Table 8. Cross-model quantization-only averages on representative LongBench tasks. Δ is “JSQKV quantizer minus KIVI”.

Model	Protocol	KIVI 4-bit	JSQKV 4-bit	Δ	KIVI 3-bit	JSQKV 3-bit	Δ	KIVI 2-bit	JSQKV 2-bit	Δ
Llama-3-8B-Instruct	representative LongBench tasks	39.07	39.13	+0.06	39.58	38.60	-0.98	25.83	28.39	+2.56
Llama-2-7B	representative LongBench tasks	28.09	28.33	+0.24	28.23	27.82	-0.41	24.33	24.17	-0.16
Mistral-7B-v0.1	representative LongBench tasks	31.19	30.78	-0.41	30.16	29.52	-0.64	23.91	25.72	+1.81

quantization, and group size 128, whereas the JSQKV quantizer uses per-token-tile quantization with tile-wise Hadamard rotation (tile size 64 and Hadamard group size 64).

Table 7 reports quantization-only results on representative LongBench tasks for Llama-3-8B-Instruct [8]. At 4-bit and 3-bit, the two methods remain close in average score. At 2-bit, however, the JSQKV quantizer is noticeably stronger, improving the average over the representative tasks from 25.83 to 28.39, with especially visible gains on HotpotQA and LCC. This pattern suggests that the tile-aligned design is most helpful in the aggressive low-bit regime.

Table 8 summarizes the same comparison on additional model families using representative LongBench tasks. It is intended to show whether the quantization trend transfers under comparable representative-task settings, rather than to support strict cross-row ranking under an identical task set. The JSQKV quantizer remains competitive on Llama-2-7B and becomes favorable at 2-bit on Mistral-7B-v0.1, suggesting that its main benefit is improved robustness under aggressive low-bit quantization.

Acknowledgments. This work was supported in part by the Natural Science Foundation of China (62372254), the Innovative Development Joint Fund Key Projects of Shandong NSF (ZR2023LZH003), and the Beijing-Tianjin-Hebei Basic Research Cooperation Project of Hebei Natural Science Foundation under Grant F2024203115 (24JCZXJC00160 in Tianjin).

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

Use of Artificial Intelligence Tools. The technical solution and experimental design presented in this paper were independently completed by the authors. AI tools were used solely for language polishing and formatting optimization, and did not participate in the development of the research ideas or core content.

References

1. Agrawal, A., Kedia, N., Panwar, A., et al.: Taming throughput-latency tradeoff in llm inference with sarathi-serve. arXiv preprint arXiv:2403.02310 (2024), <https://arxiv.org/abs/2403.02310>
2. Ashkboos, S., Mohtashami, A., Croci, M.L., Li, B., Cameron, P., Jaggi, M., Alistarh, D., Hoeffler, T., Hensman, J.: Quarot: Outlier-free 4-bit inference in rotated llms. arXiv preprint arXiv:2404.00456 (2024)
3. Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., Li, J.: Longbench: A bilingual, multitask benchmark for long context understanding. arXiv preprint arXiv:2308.14508 (2023), <https://arxiv.org/abs/2308.14508>
4. Cai, Z., Zhang, Y., Gao, B., Liu, Y., Li, Y., Liu, T., Lu, K., Xiong, W., Dong, Y., Hu, J., Xiao, W.: Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. arXiv preprint arXiv:2406.02069 (2024), <https://arxiv.org/abs/2406.02069>
5. Chen, Z., Yuan, K., Wei, H., Li, L., Shen, W., Su, Z., Yu, H.: Rotatekv: Accurate and robust 2-bit kv cache quantization for llms via outlier-aware adaptive rotations (2025), <https://arxiv.org/abs/2501.16383>
6. Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Tworek, J., Hilton, J., Nakano, R., Hesse, C., Schulman, J.: Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168 (2021), <https://arxiv.org/abs/2110.14168>
7. Ding, Y., Zhang, L.L., Zhang, C., Xu, Y., Shang, N., Xu, J., Yang, F., Yang, M.: Longrope: Extending llm context window beyond 2 million tokens. arXiv preprint arXiv:2402.13753 (2024), <https://arxiv.org/abs/2402.13753>
8. Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., et al.: The llama 3 herd of models. arXiv preprint arXiv:2407.21783 (2024)
9. Hong, K., Dai, G., Xu, J., Mao, Q., Li, X., Liu, J., Chen, K., Dong, Y., Wang, Y.: FlashDecoding++: Faster large language model inference with asynchronization, flat gemm optimization, and heuristics. In: Proceedings of Machine Learning and Systems (2024)
10. Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M.W., Shao, Y.S., Keutzer, K., Gholami, A.: Kvquant: Towards 10 million context length llm inference with kv cache quantization. arXiv preprint arXiv:2401.18079 (2024)
11. Jiang, A.Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D.S., Casas, D.d.l., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., et al.: Mistral 7b. arXiv preprint arXiv:2310.06825 (2023), <https://arxiv.org/abs/2310.06825>
12. Joo, D., Hosseini, H., Hadidi, R., Asgari, B.: Coruscant: Co-designing gpu kernel and sparse tensor core to advocate unstructured sparsity in efficient llm inference. In: Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture. pp. 232–245 (2025). <https://doi.org/10.1145/3725843.3756065>
13. Joo, D., Hosseini, H., Hadidi, R., Asgari, B.: Mustafar: Promoting unstructured sparsity for kv cache pruning in llm inference. arXiv preprint arXiv:2505.22913 (2025), <https://arxiv.org/abs/2505.22913>
14. Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I.: Efficient memory management for large language model serving with pagedattention. In: Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles (2023), <https://arxiv.org/abs/2309.06180>

15. Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., Chen, D.: Snapkv: Llm knows what you are looking for before generation. arXiv preprint arXiv:2404.14469 (2024), <https://arxiv.org/abs/2404.14469>, version 2, June 17, 2024
16. Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., Hu, X.: Kivi: A tuning-free asymmetric 2bit quantization for kv cache. arXiv preprint arXiv:2402.02750 (2024)
17. Lv, B., Zhou, Q., Ding, X., Wang, Y., Ma, Z.: Kvpruner: Structural pruning for faster and memory-efficient large language models. arXiv preprint arXiv:2409.11057 (2024), <https://arxiv.org/abs/2409.11057>
18. Merity, S., Xiong, C., Bradbury, J., Socher, R.: Pointer sentinel mixture models. arXiv preprint arXiv:1609.07843 (2016), <https://arxiv.org/abs/1609.07843>
19. Singhanian, P., Singh, S., He, S., et al.: Loki: Low-rank keys for efficient sparse attention. arXiv preprint arXiv:2406.02542 (2024), <https://arxiv.org/abs/2406.02542>
20. Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al.: Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288 (2023), <https://arxiv.org/abs/2307.09288>
21. Trivedi, H., Balasubramanian, N., Khot, T., Sabharwal, A.: Musique: Multihop questions via single-hop question composition. Transactions of the Association for Computational Linguistics **10**, 539–554 (2022), <https://aclanthology.org/2022.tacl-1.31/>
22. Xia, H., Zheng, Z., Li, Y., Zhuang, D., Zhou, Z., Qiu, X., Li, Y., Lin, W., Song, S.L.: Flash-LLM: Enabling cost-effective and highly-efficient large generative model inference with unstructured sparsity. arXiv preprint arXiv:2309.10285 (2023), <https://arxiv.org/abs/2309.10285>
23. Xiao, G., Tian, Y., Chen, B., Han, S., Lewis, M.: Efficient streaming language models with attention sinks. arXiv preprint arXiv:2309.17453 (2024), <https://arxiv.org/abs/2309.17453>
24. Xu, Y., Jie, Z., Dong, H., Wang, L., Lu, X., Zhou, A., Saha, A., Xiong, C., Sahoo, D.: Think: Thinner key cache by query-driven pruning. In: International Conference on Learning Representations (2025), <https://arxiv.org/abs/2407.21018>, spotlight
25. Yang, A., Yang, B., Hui, B., Zheng, B., Yu, B., Li, C., Zhang, D., Liu, F., Huang, H., Wei, H., et al.: Qwen2.5 technical report. arXiv preprint arXiv:2412.15115 (2024), <https://arxiv.org/abs/2412.15115>
26. Ye, Z., Chen, L., Lai, R., Lin, W., Zhang, Y., Wang, S., Chen, T., Kasikci, B., Grover, V., Krishnamurthy, A., Ceze, L.: Flashinfer: Efficient and customizable attention engine for llm inference serving. In: Proceedings of Machine Learning and Systems (2025)
27. Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Re, C., Barrett, C., Wang, Z., Chen, B.: H₂O: Heavy-hitter oracle for efficient generative inference of large language models. arXiv preprint arXiv:2306.14048 (2023), <https://arxiv.org/abs/2306.14048>
28. Zhong, Y., Liu, S., Chen, J., et al.: Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. In: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24) (2024), <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>